Lecture 17: Polynomial Interpolation; Splines

Math 6610

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Today's Goals

- Interpolation: monomials and Lagrange; existence and uniqueness of the interpolating polynomial
- Why one high degree polynomial is a bad idea (conditioning, Runge)
- Piecewise polynomials and splines of degree 1, 2, 3
- Counting unknowns and constraints: what is left free
- Cubic splines: natural, fixed-slope, not-a-knot, periodic
- The cubic spline is the smoothest interpolant

Interpolation

definition

Given $n+1$ data points

$$(x_i, y_i), \qquad i = 0, 1, \ldots, n, \qquad x_0 < x_1 < \cdots < x_n, \tag{1}$$

a function $p(x)$ *interpolates* the data if $p(x_i) = y_i$ for $i = 0, 1, \ldots, n$. The x_i are the *nodes*.

Where it is used:

- values between measurements: readings every hour, needed at 2:15
- resampling: resizing images, changing audio sample rates (Matlab `interp1`, `interp2`)
- inside other methods: integrate or differentiate the interpolant (quadrature, Lecture 23; ODE solvers)

We want a "simple" $p(x)$: cheap to evaluate, differentiate, integrate. First choice: polynomials. Then: piecewise polynomials (splines).

Recall: Monomials

Obvious attempt: try picking

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

So for each x_i we have

$$p(x_i) = a_0 + a_1 x_i + a_2 x_i^2 + \cdots + a_n x_i^n = y_i$$

OR

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ & & & \vdots & \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

That is,

$$a = M^{-1} y$$

Very bad matrix: terribly ill-conditioned, inverse entries are **large**
Very bad evaluation: values are **huge**

Monomials: Is M Nonsingular?

definition

$\prod_n$ is the set of all polynomials of degree at most n:

$$p(x) = a_0 + a_1 x + \cdots + a_n x^n, \qquad a_0, \ldots, a_n \in \mathbb{R} \qquad (n+1 \text{ coefficients}).$$

- $p(x_i) = y_i$, $i = 0, \ldots, n$: $n+1$ linear equations for $n+1$ unknowns $a_0, \ldots, a_n$.
- The matrix M (previous slide) is the *Vandermonde matrix*.
- Square system: solve by LU with partial pivoting (Lecture 16).
- Two questions: is M nonsingular (invertible)? How well conditioned is it?

The first answer comes from the Lagrange form.

Lagrange

The general Lagrange form is

$$\ell_k(x) = \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}$$

The resulting interpolating polynomial is

$$p(x) = \sum_{k=0}^{n} \ell_k(x) y_k$$

Lagrange: Existence and Uniqueness

Theorem: Polynomial Interpolation

Let $x_0 < x_1 < \cdots < x_n$ and $y_0, \ldots, y_n \in \mathbb{R}$. There is exactly one $p_n \in \prod_n$ that interpolates the data (1), namely

$$p_n(x) = \sum_{k=0}^{n} \ell_k(x)\, y_k, \qquad \ell_k(x) = \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}.$$

Equivalently, the monomial matrix M is nonsingular.

- ℓ_k is 1 at x_k and 0 at the other nodes.
- So in the Lagrange basis the "matrix" is the identity: no solve.

Lagrange: Proof

Proof. Idea: check the values of ℓ_k at the nodes; uniqueness by counting zeros.
1. $\ell_k \in \prod_n$ *and* $\ell_k(x_j) = \delta_{kj}$ (1 if $j = k$, else 0). n linear factors; denominators $\neq 0$. For $j \neq k$ the factor $i = j$ vanishes; for $j = k$ all factors are 1.
2. Interpolation. For every $j = 0, \ldots, n$:

$$p_n(x_j) = \sum_{k=0}^{n} \ell_k(x_j)\, y_k = y_j.$$

3. Uniqueness. If $q \in \prod_n$ also interpolates, then $d := p_n - q \in \prod_n$ is 0 at all $n+1$ nodes. A nonzero polynomial of degree $\leqslant n$ has at most n zeros, so $d \equiv 0$.
4. M *nonsingular.* $Ma = 0$ means: coefficients a interpolate the data $y = 0$. So does 0; by 3., $a = 0$. ■

Example

Find the equation of the parabola passing through the points (1,6), (-1,0), and (2,12)

$x_0 = 1, x_1 = -1, x_2 = 2; \qquad y_0 = 6, y_1 = 0, y_2 = 12;$

$$\begin{aligned}
\ell_0(x) &= \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} = \frac{(x+1)(x-2)}{(2)(-1)} \\
\ell_1(x) &= \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} = \frac{(x-1)(x-2)}{(-2)(-3)} \\
\ell_2(x) &= \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} = \frac{(x-1)(x+1)}{(1)(3)}
\end{aligned}$$

$$\begin{aligned}
p_2(x) &= y_0\ell_0(x) + y_1\ell_1(x) + y_2\ell_2(x) \\
&= -3 \times (x+1)(x-2) + 0 \times \frac{1}{6}(x-1)(x-2) \\
&\quad +4 \times (x-1)(x+1) \\
&= (x+1)[4(x-1) - 3(x-2)] \\
&= (x+1)(x+2)
\end{aligned}$$

Example: Three Points, Two Ways

Example. Data $(0,1)$, $(1,3)$, $(2,2)$, so $n = 2$.
Lagrange:

$$\ell_0 = \frac{(x-1)(x-2)}{2}, \quad \ell_1 = \frac{x(x-2)}{-1}, \quad \ell_2 = \frac{x(x-1)}{2}, \qquad p_2 = 1 \cdot \ell_0 + 3\,\ell_1 + 2\,\ell_2.$$

Expanding: $p_2(x) = 1 + \frac{7}{2}x - \frac{3}{2}x^2$.
Monomials:

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} \Rightarrow\ a_0 = 1,\ a_1 + a_2 = 2,\ 2a_1 + 4a_2 = 1 \ \Rightarrow\ a = \left(1, \tfrac{7}{2}, -\tfrac{3}{2}\right)^T.$$

Same polynomial, as the theorem says. (Check: $p_2(1) = 1 + 3.5 - 1.5 = 3$, $p_2(2) = 1 + 7 - 6 = 2$.)

Monomials in Practice

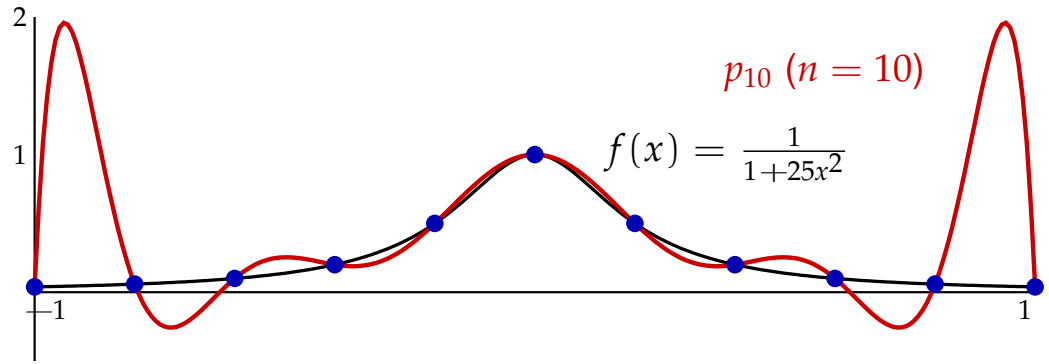
Conditioning of M, equally spaced nodes (computed, double precision):

n	5	10	15	20
$\kappa_2(M)$, nodes in $[-1,1]$	$6.4 \cdot 10^1$	$1.4 \cdot 10^4$	$3.3 \cdot 10^6$	$8.3 \cdot 10^8$
$\kappa_2(M)$, nodes in $[0,1]$	$4.9 \cdot 10^3$	$1.2 \cdot 10^8$	$3.1 \cdot 10^{12}$	$8.9 \cdot 10^{16}$

- $\kappa_2(M)$ grows exponentially in n. By Lecture 15, the computed a can lose about $\log_{10} \kappa(M)$ digits: on $[0,1]$ with $n = 20$, possibly all 16.
- Matlab `polyfit(x,y,n)` solves this system; for large n it warns that the polynomial is badly conditioned.
- Few points (say $n \leqslant 5$): any form is fine.
- Many points: do not use one high degree polynomial on equally spaced nodes (next slide). Use splines (today) or Chebychev nodes (Lecture 18).

How Bad Is Polynomial Interpolation?

Example. Runge's example, equally spaced nodes on $[-1, 1]$:



Interpolate $f(x) = 1/(1 + 25x^2)$ at $x_i = -1 + 2i/n$. Maximum error on $[-1, 1]$ (computed):

n	5	10	20
$\max \lvert f - p_n \rvert$	0.43	1.92	59.8

The error grows with n, although f is very smooth. Why, and how to fix it: the error formula in Lecture 18.

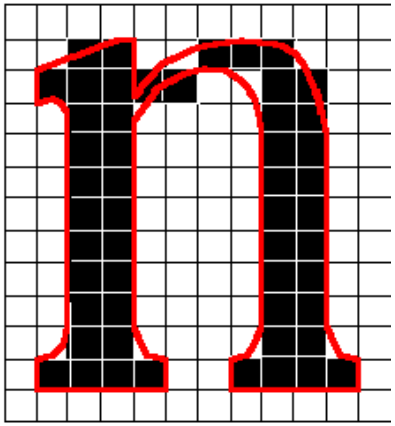
From One Polynomial to Splines

Two problems with one polynomial p_n through all the data:

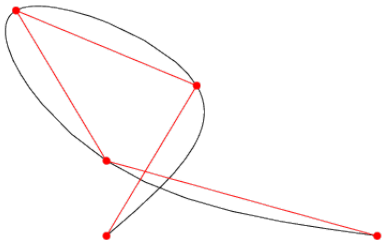
- **oscillation:** high degree on equally spaced nodes can oscillate wildly near the ends (Runge)
- **global coupling:** changing one y_k changes p_n on all of $[x_0, x_n]$ (it adds $\delta y_k \, \ell_k(x)$)

Fix: low, fixed degree on each subinterval, glued together smoothly: *piecewise polynomials*.

Why Splines?







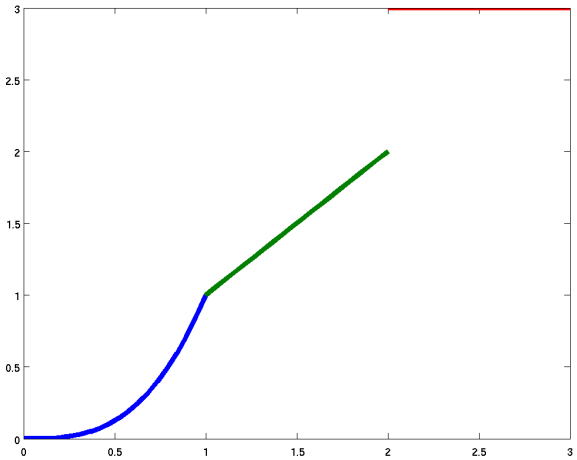
- truetype fonts, postscript, metafonts
- graphics surfaces
- smooth surfaces are needed
- how do we interpolate smoothly a set of data?
- keywords: Bezier Curves, splines, B-splines, NURBS
- basic tool: piecewise interpolation

Piecewise Polynomial

A function $f(x)$ is considered a piecewise polynomial on $[a, b]$ if there exists a (finite) partition P of $[a, b]$ such that $f(x)$ is a polynomial on each $[t_i, t_{i+1}] \in P$.

Example.

$$f(x) = \begin{cases} x^3 & x \in [0, 1] \\ x & x \in (1, 2) \\ 3 & x \in [2, 3] \end{cases}$$

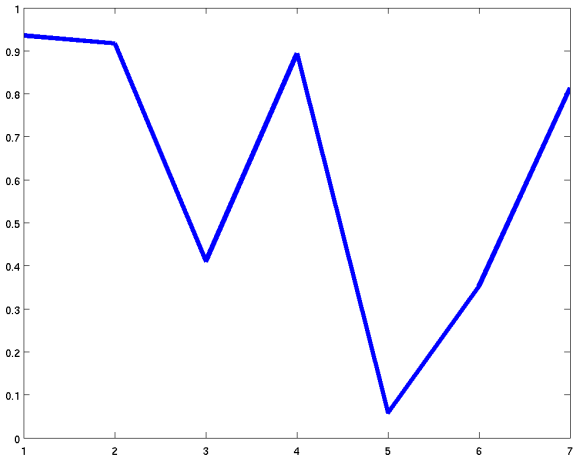


What do we want?

- we would like the piecewise polynomial to do two things
  1. interpolate (or be close to) some set of data points
  2. look nice (smooth)
- one option is called a *spline*

splines

- A *spline* is a piecewise polynomial with a certain level of smoothness.
- take Matlab: `plot(1:7,rand(7,1))`
- this is linear and continuous, but not very smooth
- the function changes behavior at *knots* $t_0, \ldots, t_n$



spline of degree k

definition

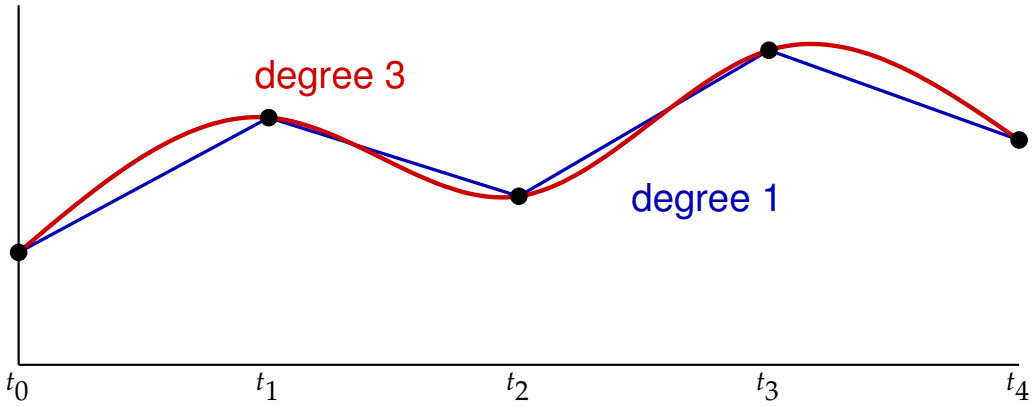
A function $S(x)$ is a spline of degree k ($k \geqslant 1$) if:

1. The domain of $S(x)$ is an interval $[a, b]$
2. $S(x), S'(x), \ldots, S^{(k-1)}(x)$ are continuous on $[a, b]$
3. There is a partition $a = t_0 < t_1 < \cdots < t_n = b$ such that on each subinterval $[t_i, t_{i+1}]$, $S(x) = S_i(x)$ is a polynomial of degree at most k.

- The t_i are the *knots*; S_i is the piece on $[t_i, t_{i+1}]$, $i = 0, \ldots, n-1$.
- $k = 1, 2, 3$: the degree 1, 2, 3 (cubic) splines on the next slides.
- Data at the knots: (t_i, y_i), $i = 0, \ldots, n$; we write $h_i = t_{i+1} - t_i > 0$.

degree 1 vs degree 3 spline

Example. The same five data points:



- degree 1: continuous, corners at the knots
- degree 3: S, S', S'' continuous, so no visible corners (the cubic shown is the natural cubic spline)

degree 1 spline

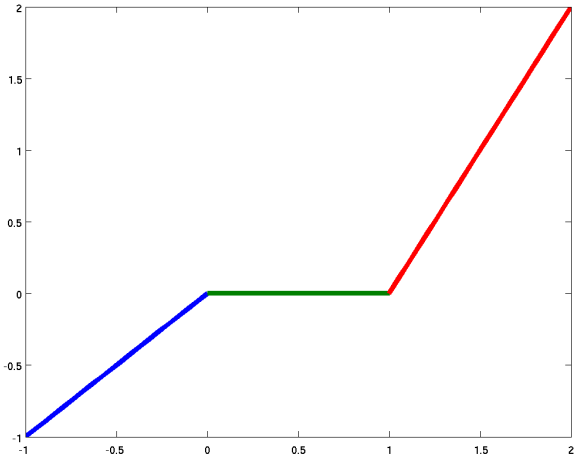
definition

A function $S(x)$ is a spline of degree 1 if:

1. The domain of $S(x)$ is an interval $[a, b]$
2. $S(x)$ is continuous on $[a, b]$
3. There is a partition $a = t_0 < t_1 < \cdots < t_n = b$ such that $S(x)$ is linear on each subinterval $[t_i, t_{i+1}]$.

Example.

$$S(x) = \begin{cases} x & x \in [-1, 0] \\ 1 & x \in (0, 1) \\ 2x - 2 & x \in [1, 2] \end{cases}$$



degree 1 spline

Given data $t_0, \ldots, t_n$ and $y_0, \ldots, y_n$, how do we form a spline?
We need two things to hold in the interval $[a, b] = [t_0, t_n]$:

1. $S(t_i) = y_i$ for $i = 0, \ldots, n$
2. $S_i(x) = a_i x + b_i$ for $i = 0, \ldots, n$

Write $S_i(x)$ in point-slope form

$$\begin{aligned} S_i(x) &= y_i + m_i(x - t_i) \\ &= y_i + \frac{y_{i+1} - y_i}{t_{i+1} - t_i}(x - t_i) \end{aligned}$$

Done.

degree 1 spline

```
1  input t,y vectors of data
2  input evaluation location x
3  find interval i with x ∈ [t_i, t_{i+1}]
4  S = y_i + (x-t_i) m_i
```

degree 1 spline

Interesting:

- input $n+1$ data points $t_0, \ldots, t_n, y_0, \ldots, y_n$
- in each interval we have $S_i(x) = a_i x + b_i$
- 2 unknowns per interval $[t_i, t_{i+1}]$
- or $2n$ total unknowns
- the $n+1$ pieces of input contraints $S(t_i) = y_i$ gives 2 constraints per interval
- or $2n$ total constraints

degree 1 spline: existence and uniqueness

Proposition

For data (t_i, y_i), $i = 0, \ldots, n$, with $t_0 < \cdots < t_n$, there is exactly one spline $S(x)$ of degree 1 with $S(t_i) = y_i$ for all i. On $[t_i, t_{i+1}]$, $i = 0, \ldots, n-1$, it is the point-slope form

$$S_i(x) = y_i + \frac{y_{i+1} - y_i}{t_{i+1} - t_i}(x - t_i).$$

Proof. *1. Each piece.* S_i is linear with $S_i(t_i) = y_i$ and $S_i(t_{i+1}) = y_{i+1}$. Two points give exactly one line (Lagrange, $n = 1$): S_i is the formula.
2. Gluing. $S_{i-1}(t_i) = y_i = S_i(t_i)$, $i = 1, \ldots, n-1$, so S is continuous. Nothing else is required. ∎

- Matlab: `interp1(t, y, x)`. Finding i: binary search, $O(\log n)$.
- Changing y_i changes S only on $[t_{i-1}, t_{i+1}]$: local.

degree 2 splines

definition

A function $S(x)$ is a spline of degree 2 if:

1. The domain of $S(x)$ is an interval $[a, b]$
2. $S(x)$ is continuous on $[a, b]$
3. $S'(x)$ is continuous on $[a, b]$
4. There is a partition $a = t_0 < t_1 < \cdots < t_n = b$ such that $S(x)$ is quadratic on each subinterval $[t_i, t_{i+1}]$.

degree 2 splines

$$S(x) = \begin{cases} S_0(x) & x \in [t_0, t_1] \\ S_1(x) & x \in [t_1, t_2] \\ \vdots & \vdots \\ S_{n-1}(x) & x \in [t_{n-1}, t_n] \end{cases}$$

for each i we have

$$S_i(x) = a_i x^2 + b_i x + c_i$$

What are a_i, b_i, c_i?

degree 2 splines

- 3 unknowns in each interval
- $3n$ total unknowns
- $2n$ constraints for matching up the input data (2 per interval)
- $n-1$ interior points require continuity of the derivative: $S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$
- but this is just $n-1$ constraints
- total of $3n-1$ constraints
- extra constraint: $S'(x_0) =$given, for example.

degree 3 spline: cubic spline

definition

A function $S(x)$ is a spline of degree 3 if:

1. The domain of $S(x)$ is an interval $[a, b]$
2. $S(x)$ is continuous on $[a, b]$
3. $S'(x)$ is continuous on $[a, b]$
4. $S''(x)$ is continuous on $[a, b]$
5. There is a partition $a = t_0 < t_1 < \cdots < t_n = b$ such that $S(x)$ is cubic on each subinterval $[t_i, t_{i+1}]$.

degree 3 spline: cubic spline

In each interval $[t_i, t_{i+1}]$, $S(x)$ looks like

$$S_i(x) = a_{0,i} + a_{1,i}x + a_{2,i}x^2 + a_{3,i}x^3$$

- n intervals, $n+1$ knots, 4 unknowns per interval
- $4n$ unknowns
- $2n$ constraints by $S(t_i)$, $S(t_{i+1})$ specified (continuity of S)
- $n-1$ constraints by continuity of $S'(x)$
- $n-1$ constraints by continuity of $S''(x)$
- $4n-2$ total constraints

This leaves 2 extra degrees of freedom. The cubic spline is not yet unique!

spline of degree k: counting

In each interval $[t_i, t_{i+1}]$, $S_i(x)$ has $k+1$ coefficients:

unknowns: n intervals $\times$ $(k+1)$	$n(k+1)$
$S(t_i)$, $S(t_{i+1})$ specified (2 per interval)	$2n$
continuity of $S', \ldots, S^{(k-1)}$ at $t_1, \ldots, t_{n-1}$	$(n-1)(k-1)$

$n(k+1) - 2n - (n-1)(k-1) = k-1$ extra degrees of freedom.

- degree 1: 0 extra; degree 2: 1 extra; degree 3: 2 extra (as on the previous slides).
- All constraints are linear in the coefficients. The count suggests, but does not prove, that $k-1$ more constraints give a unique spline: the equations could be dependent. Existence and uniqueness need a proof for each case.

degree 3 spline: cubic spline

Some options:

- natural cubic spline: $S''(t_0) = S''(t_n) = 0$
- fixed-slope: $S'(t_0) = a$, $S'(t_n) = b$
- not-a-knot: $S'''(x)$ continuous at t_1 and t_{n-1}
- periodic: S' and S'' are periodic at the ends

cubic spline: the end conditions precisely

With S_i the cubic on $[t_i, t_{i+1}]$, $i = 0, \ldots, n-1$:

- *natural cubic spline:* $S_0''(t_0) = 0$ and $S_{n-1}''(t_n) = 0$ (no curvature at the ends)
- *fixed-slope:* $S_0'(t_0) = a$ and $S_{n-1}'(t_n) = b$, with given end slopes $a, b \in \mathbb{R}$
- *not-a-knot* (needs $n \geqslant 3$): $S_0'''(t_1) = S_1'''(t_1)$ and $S_{n-2}'''(t_{n-1}) = S_{n-1}'''(t_{n-1})$
- *periodic* (needs $y_0 = y_n$): $S_0'(t_0) = S_{n-1}'(t_n)$ and $S_0''(t_0) = S_{n-1}''(t_n)$

Each gives the 2 missing constraints: $4n - 2 + 2 = 4n$ equations for $4n$ unknowns.

Careful: in the fixed-slope condition, a and b are slopes, not the ends of the interval $[t_0, t_n]$.

cubic spline: why “not-a-knot”?

Proposition

With the not-a-knot condition, $S_0 = S_1$ (as polynomials), so $S(x)$ is a single cubic on $[t_0, t_2]$; likewise $S_{n-2} = S_{n-1}$ on $[t_{n-2}, t_n]$. So t_1 and t_{n-1} are not really knots.

Proof. Idea: a cubic is fixed by its value and three derivatives at one point.

1. *Matching at* t_1. $S_0^{(m)}(t_1) = S_1^{(m)}(t_1)$ for $m = 0, 1, 2$ (S, S', S'' continuous) and $m = 3$ (the condition).
2. *Taylor at* t_1 is exact for cubics:

$$S_l(x) = \sum_{m=0}^{3} \frac{S_l^{(m)}(t_1)}{m!}(x - t_1)^m, \quad l = 0, 1 \quad \Longrightarrow \quad S_0 = S_1.$$

3. *At* t_{n-1}*:* the same argument with S_{n-2}, S_{n-1}. ∎

cubic spline: existence and uniqueness

Theorem

Let $a = t_0 < \cdots < t_n = b$ and $y_0, \ldots, y_n \in \mathbb{R}$. There is exactly one cubic spline $S(x)$ with $S(t_i) = y_i$, $i = 0, \ldots, n$, and

- the natural or the fixed-slope condition: for every $n \geqslant 1$;
- the periodic condition: for every $n \geqslant 1$, if $y_0 = y_n$;
- the not-a-knot condition: for every $n \geqslant 3$.

- Proof for the natural cubic spline: Lecture 18 (the constraints become a nonsingular tridiagonal system). Other cases: similar linear systems; proof omitted.
- In words: pick the end condition, and the data determine the curve. The work is one linear solve.

cubic spline: which end condition?

end condition	needs	Matlab
not-a-knot	data only	`spline(t,y)` (default)
fixed-slope	end slopes a, b	`spline(t,[a y b])`
natural	data only	`csape(t,y,'variational')`
periodic	$y_0 = y_n$	`csape(t,y,'periodic')`

(`csape`: Curve Fitting Toolbox. Evaluate with `ppval(pp,x)`.)

- **not-a-knot:** uses no extra information; good default
- **fixed-slope:** use when the true end slopes are known (e.g. a velocity at the endpoints); then it is the most accurate (Lecture 18)
- **natural:** simplest to analyze, but it forces $S'' = 0$ at the ends; if $f''(t_0) \neq 0$ or $f''(t_n) \neq 0$, accuracy near the ends drops (Lecture 18)
- **periodic:** closed curves, periodic signals (angles, daily cycles)

The Smoothest Interpolant

definition

For g continuous on $[a, b]$, the L^2 norm is

$$\|g\|_2 = \left(\int_a^b |g(x)|^2 \, dx \right)^{1/2} \qquad \text{(the continuous analogue of the vector 2-norm)}.$$

Theorem 17.7: Minimum Curvature

Let $S(x)$ be a cubic spline with $S(t_i) = y_i$, $i = 0, \ldots, n$. Let f have f'' continuous on $[a, b]$ and $f(t_i) = y_i$, $i = 0, \ldots, n$. If

$$S''(x)\big[f'(x) - S'(x)\big]\Big|_{x=a}^{x=b} = 0 \qquad \text{(boundary identity)},$$

then $\|S''\|_2 \leqslant \|f''\|_2$, with equality only if $f = S$.

Among all such curves through the data, the cubic spline has the least total squared curvature $\int_a^b (f'')^2 \, dx$: it is the "flattest". Proof: Lecture 18.

When Does the Boundary Identity Hold?

Proposition

$S''(f' - S')\big|_a^b = 0$ holds in these cases:

- S natural, any f: $S''(a) = S''(b) = 0$.
- S fixed-slope, f with the same end slopes $f'(a) = S'(a), f'(b) = S'(b)$: $f' - S' = 0$ at both ends.
- S periodic, f with $f'(a) = f'(b)$: using $S''(a) = S''(b)$ and $S'(a) = S'(b)$,

$$S''(b)\big[f'(b) - S'(b)\big] - S''(a)\big[f'(a) - S'(a)\big] = S''(a)\big[f'(b) - f'(a)\big] = 0.$$

- not-a-knot: neither S'' nor $f' - S'$ need vanish at the ends; Theorem 17.7 is not guaranteed.
- natural cubic spline: flattest among *all* interpolants. Fixed-slope: flattest among interpolants with the same end slopes.

Why “Spline”?

A draftsman’s spline is a thin flexible strip held by weights at the data points. Its bending energy is proportional to

$$\int \kappa_c^2 \, ds, \qquad \kappa_c = \frac{f''}{(1+(f')^2)^{3/2}}, \quad ds = \sqrt{1+(f')^2}\, dx$$

(curvature of the graph, integrated over arc length).

- For small slopes, $|f'| \ll 1$, this is about $\int_a^b (f'')^2 \, dx$: the quantity minimized in Theorem 17.7.
- The free ends of the strip carry no bending moment: $S'' = 0$, the natural cubic spline.
- The proof (next time) is one integration by parts plus Pythagoras: $f'' - S''$ is orthogonal to S''. The same idea is behind least squares and finite element methods.

Next time...

- interpolation error, Chebychev nodes
- constructing the natural cubic spline